Python Code

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re
import warnings
warnings.filterwarnings('ignore')

# sklearn stuff
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder, OrdinalEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, roc_auc_score, roc_curve,
                             confusion_matrix, mean_absolute_error, r2_score)
from sklearn.inspection import permutation_importance

try:
    import shap
    shap_available = True
except ImportError:
    shap_available = False
    print("SHAP not installed, skipping SHAP plots. Run: pip install shap")

sns.set_theme(style='whitegrid', palette='muted')

# ==============================================================================
# STEP 1 - LOAD AND CLEAN THE DATA
# ==============================================================================

print("Loading the dataset...")
df = pd.read_csv('Food Coma Study_ Why do we feel sleepy after eating_ (Responses) - Form Responses 1.csv')
print("Dataset shape:", df.shape)
print("\nMissing values:")
print(df.isnull().sum()[df.isnull().sum() > 0])

# dropping columns we don't need
df = df.drop(columns=['Timestamp', 'Email Address',
                      'If you had a nap, what was the duration?',
                      'If yes, after how long do you feel drowsy?'])

# --- clean Age column (has messy values like "22+", "Twenty Years", "Soyal") ---
def clean_age(val):
    if pd.isnull(val):
```

```python
        return np.nan
    # just try to find any number in the string
    match = re.search(r'\d+', str(val))
    if match:
        return int(match.group())
    return np.nan

df['Age'] = df['Age'].apply(clean_age)
df['Age'] = df['Age'].fillna(df['Age'].median()).astype(int)
print("\nAge cleaned. Range:", df['Age'].min(), "-", df['Age'].max())

# --- clean Height column (has cm, feet, decimal feet all mixed) ---
def clean_height(val):
    if pd.isnull(val):
        return np.nan
    s = str(val).strip().lower()
    s = s.replace('cm', '').replace('feet', '').replace('foot', '').strip()
    # handle feet + inches format like 5'3" or 5'4
    if "'" in s:
        parts = re.split(r"['\"]", s)
        try:
            feet = float(parts[0].strip())
            inches = float(parts[1].strip()) if len(parts) > 1 and parts[1].strip() else 0
            return round(feet * 30.48 + inches * 2.54, 1)
        except:
            return np.nan
    try:
        num = float(s)
        if 4.0 <= num <= 7.9:   # it's in decimal feet like 5.7
            return round(num * 30.48, 1)
        elif num >= 100:          # already in cm
            return round(num, 1)
        else:
            return np.nan         # invalid like "7.5 cm"
    except:
        return np.nan

df['Height_cm'] = df['Height (cm)'].apply(clean_height)
df = df.drop(columns=['Height (cm)'])
df['Height_cm'] = df['Height_cm'].fillna(df['Height_cm'].median())
print("Height cleaned. Range:", df['Height_cm'].min(), "-", df['Height_cm'].max(), "cm")

# --- clean Weight column ---
def clean_weight(val):
    if pd.isnull(val):
        return np.nan
    try:
        return float(str(val).lower().replace('kg', '').replace('~', '').strip())
    except:
        return np.nan

df['Weight_kg'] = df['Weight (kg)'].apply(clean_weight)
```

```python
df = df.drop(columns=['Weight (kg)'])
df['Weight_kg'] = df['Weight_kg'].fillna(df['Weight_kg'].median())
print("Weight cleaned. Range:", df['Weight_kg'].min(), "-", df['Weight_kg'].max(), "kg")

# calculate BMI since we have height and weight now
df['BMI'] = (df['Weight_kg'] / (df['Height_cm'] / 100) ** 2).round(1)
print("BMI calculated. Range:", df['BMI'].min(), "-", df['BMI'].max())

# fill missing values in other columns
df['Physical activity before meal (last 2\u20133 hrs):'] = \
    df['Physical activity before meal (last 2\u20133 hrs):'].fillna('Missing')
df['Do you regularly feel sleepy after meals?'] = \
    df['Do you regularly feel sleepy after meals?'].fillna('No')
df['Do you consider yourself:'] = \
    df['Do you consider yourself:'].fillna('Morning person')

# make the drowsiness rating numeric
df['Rate your drowsiness'] = pd.to_numeric(df['Rate your drowsiness'], errors='coerce')

print("\nCleaning done! Final shape:", df.shape)
print("Any remaining nulls:", df.isnull().sum().sum())

# ==============================================================================
# STEP 2 - EDA (only for the features we will use in the model)
# ==============================================================================

print("\nDoing Exploratory Data Analysis on selected features...")

# the 8 features we picked for our model + the target column
selected_features = [
    'Meal Size',
    'Carb Content',
    'Protein Content',
    'How would you rate your sleep quality? ',
    'What was the last meal type you had?',
    'Physical activity before meal (last 2\u20133 hrs):',
    'How heavy did the meal feel?',
    'Stress level before eating'
]

target = 'Did you feel drowsy after your meal?'

# --- EDA Plot 1: How many people felt drowsy? ---
plt.figure(figsize=(6, 5))
counts = df[target].value_counts()
colors = ['#E57373', '#81C784']
bars = plt.bar(counts.index, counts.values, color=colors, edgecolor='black', width=0.5)
for bar, val in zip(bars, counts.values):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.5,
             str(val), ha='center', fontsize=12, fontweight='bold')
plt.title('How Many People Felt Drowsy After Eating?', fontsize=14, fontweight='bold')
```

```python
plt.ylabel('Number of Respondents')
plt.xlabel('Felt Drowsy?')
plt.tight_layout()
plt.show()

# --- EDA Plot 2: All 8 selected features vs target (2 rows x 4 cols) ---
fig, axes = plt.subplots(2, 4, figsize=(22, 10))
fig.suptitle('Selected Features vs Drowsiness (Target Variable)', fontsize=16, fontweight='bold')

# Meal Size
meal_size_order = ['Small', 'Medium', 'Large', 'Very Large']
meal_size_data = df.groupby('Meal Size')[target].apply(
    lambda x: (x == 'Yes').mean() * 100).reindex(meal_size_order)
axes[0, 0].bar(meal_size_order, meal_size_data.values,
               color=['#66BB6A', '#FFA726', '#EF5350', '#B71C1C'], edgecolor='black', alpha=0.85)
for i, v in enumerate(meal_size_data.values):
    axes[0, 0].text(i, v + 1, f'{v:.0f}%', ha='center', fontsize=10, fontweight='bold')
axes[0, 0].set_title('Meal Size vs % Drowsy', fontweight='bold')
axes[0, 0].set_ylabel('% Feeling Drowsy')
axes[0, 0].set_ylim(0, 110)

# Carb Content
carb_order = ['Low', 'Medium', 'High']
carb_data = df.groupby('Carb Content')[target].apply(
    lambda x: (x == 'Yes').mean() * 100).reindex(carb_order)
axes[0, 1].bar(carb_order, carb_data.values,
               color=['#66BB6A', '#FFA726', '#EF5350'], edgecolor='black', alpha=0.85)
for i, v in enumerate(carb_data.values):
    axes[0, 1].text(i, v + 1, f'{v:.0f}%', ha='center', fontsize=10, fontweight='bold')
axes[0, 1].set_title('Carb Content vs % Drowsy', fontweight='bold')
axes[0, 1].set_ylabel('% Feeling Drowsy')
axes[0, 1].set_ylim(0, 110)

# Protein Content
protein_order = ['Low', 'Medium', 'High']
protein_data = df.groupby('Protein Content')[target].apply(
    lambda x: (x == 'Yes').mean() * 100).reindex(protein_order)
axes[0, 2].bar(protein_order, protein_data.values,
               color=['#EF5350', '#FFA726', '#66BB6A'], edgecolor='black', alpha=0.85)
for i, v in enumerate(protein_data.values):
    axes[0, 2].text(i, v + 1, f'{v:.0f}%', ha='center', fontsize=10, fontweight='bold')
axes[0, 2].set_title('Protein Content vs % Drowsy', fontweight='bold')
axes[0, 2].set_ylabel('% Feeling Drowsy')
axes[0, 2].set_ylim(0, 110)

# Sleep Quality (numeric - box plot)
drowsy_yes = df[df[target] == 'Yes']['How would you rate your sleep quality? ']
drowsy_no  = df[df[target] == 'No']['How would you rate your sleep quality? ']
axes[0, 3].boxplot([drowsy_yes.dropna(), drowsy_no.dropna()],
```

```python
                    labels=['Drowsy', 'Not Drowsy'],
                    patch_artist=True,
                    boxprops=dict(facecolor='#90CAF9'),
                    medianprops=dict(color='red', linewidth=2))
axes[0, 3].set_title('Sleep Quality vs Drowsiness', fontweight='bold')
axes[0, 3].set_ylabel('Sleep Quality Rating (1-5)')

# Meal Type
meal_type_data = df.groupby('What was the last meal type you had?')[target].apply(
    lambda x: (x == 'Yes').mean() * 100).sort_values(ascending=False)
colors_mt = ['#EF5350' if v > 70 else '#FFA726' if v > 55 else '#66BB6A'
             for v in meal_type_data.values]
axes[1, 0].bar(meal_type_data.index, meal_type_data.values,
               color=colors_mt, edgecolor='black', alpha=0.85)
for i, v in enumerate(meal_type_data.values):
    axes[1, 0].text(i, v + 1, f'{v:.0f}%', ha='center', fontsize=10, fontweight='bold')
axes[1, 0].set_title('Meal Type vs % Drowsy', fontweight='bold')
axes[1, 0].set_ylabel('% Feeling Drowsy')
axes[1, 0].set_ylim(0, 110)

# Physical Activity
activity_data = df.groupby('Physical activity before meal (last 2\u20133 hrs):')[target].apply(
    lambda x: (x == 'Yes').mean() * 100).sort_values(ascending=False)
colors_act = ['#EF5350' if v > 70 else '#FFA726' if v > 55 else '#66BB6A'
              for v in activity_data.values]
axes[1, 1].bar(activity_data.index, activity_data.values,
               color=colors_act, edgecolor='black', alpha=0.85)
for i, v in enumerate(activity_data.values):
    axes[1, 1].text(i, v + 1, f'{v:.0f}%', ha='center', fontsize=9, fontweight='bold')
axes[1, 1].set_title('Pre-Meal Activity vs % Drowsy', fontweight='bold')
axes[1, 1].set_ylabel('% Feeling Drowsy')
axes[1, 1].set_ylim(0, 110)
axes[1, 1].tick_params(axis='x', labelsize=8)

# Meal Heaviness (numeric - box plot)
heavy_yes = df[df[target] == 'Yes']['How heavy did the meal feel?']
heavy_no  = df[df[target] == 'No']['How heavy did the meal feel?']
axes[1, 2].boxplot([heavy_yes.dropna(), heavy_no.dropna()],
                   labels=['Drowsy', 'Not Drowsy'],
                   patch_artist=True,
                   boxprops=dict(facecolor='#FFCC80'),
                   medianprops=dict(color='red', linewidth=2))
axes[1, 2].set_title('Meal Heaviness vs Drowsiness', fontweight='bold')
axes[1, 2].set_ylabel('Heaviness Rating (1-5)')

# Stress Level
stress_order = ['Low', 'Medium', 'High']
stress_data = df.groupby('Stress level before eating')[target].apply(
    lambda x: (x == 'Yes').mean() * 100).reindex(stress_order)
axes[1, 3].bar(stress_order, stress_data.values,
               color=['#66BB6A', '#FFA726', '#EF5350'], edgecolor='black', alpha=0.85)
for i, v in enumerate(stress_data.values):
```

```python
    axes[1, 3].text(i, v + 1, f'{v:.0f}%', ha='center', fontsize=10, fontweight='bold')
axes[1, 3].set_title('Stress Level vs % Drowsy', fontweight='bold')
axes[1, 3].set_ylabel('% Feeling Drowsy')
axes[1, 3].set_ylim(0, 110)

plt.tight_layout()
plt.show()

# ==============================================================================
# STEP 3 - PREPROCESSING PIPELINE
# ==============================================================================

print("\nSetting up the preprocessing pipeline...")

# separating features by their type so we can handle them differently
numeric_features  = ['How would you rate your sleep quality? ',
                     'How heavy did the meal feel?']

ordinal_features  = ['Meal Size', 'Carb Content', 'Protein Content',
                     'Stress level before eating']

# order matters here - Low < Medium < High etc
ordinal_categories = [
    ['Small', 'Medium', 'Large', 'Very Large'],
    ['Low', 'Medium', 'High'],
    ['Low', 'Medium', 'High'],
    ['Low', 'Medium', 'High'],
]

nominal_features = ['What was the last meal type you had?',
                    'Physical activity before meal (last 2\u20133 hrs):']

# pipeline for each type of feature
num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),   # fill missing with median
    ('scaler',  StandardScaler()),                   # scale to mean=0, std=1
])

ord_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('ordinal', OrdinalEncoder(categories=ordinal_categories,
                               handle_unknown='use_encoded_value',
                               unknown_value=-1)),
])

nom_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='constant', fill_value='Missing')),
    ('onehot',  OneHotEncoder(handle_unknown='ignore', sparse_output=False)),
])

# combine all three pipelines
preprocessor = ColumnTransformer(transformers=[
    ('num', num_pipeline, numeric_features),
```

```python
    ('ord', ord_pipeline, ordinal_features),
    ('nom', nom_pipeline, nominal_features),
])

# ==============================================================================
# STEP 4 - TRAIN TEST SPLIT
# ==============================================================================

X = df[selected_features]
y = df[target].map({'Yes': 1, 'No': 0})   # convert Yes/No to 1/0

# using stratify so both train and test have same class ratio
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=24, stratify=y
)

print(f"\nTrain size: {len(X_train)} | Test size: {len(X_test)}")
print(f"Drowsy in train: {y_train.sum()} | Not drowsy: {(y_train==0).sum()}")

# ==============================================================================
# STEP 5 - TRAIN THE MODELS
# ==============================================================================

print("\nTraining the models...")

# defining our 3 classifiers
# class_weight balanced helps since we have more drowsy than not drowsy (84 vs 43)
rf  = RandomForestClassifier(n_estimators=100, max_depth=5,
                             class_weight='balanced', random_state=42)
svm = SVC(C=1, kernel='rbf', probability=True,
          class_weight='balanced', random_state=42)
lr  = LogisticRegression(C=0.1, max_iter=1000,
                         class_weight='balanced', random_state=42)

# wrapping each model with the preprocessor in a pipeline
rf_pipe  = Pipeline(steps=[('preprocessor', preprocessor), ('classifier', rf)])
svm_pipe = Pipeline(steps=[('preprocessor', preprocessor), ('classifier', svm)])
lr_pipe  = Pipeline(steps=[('preprocessor', preprocessor), ('classifier', lr)])

rf_pipe.fit(X_train, y_train)
svm_pipe.fit(X_train, y_train)
lr_pipe.fit(X_train, y_train)

print("All models trained!")

# get predictions and probabilities for all 3 models
rf_pred    = rf_pipe.predict(X_test)
svm_pred   = svm_pipe.predict(X_test)
lr_pred    = lr_pipe.predict(X_test)

rf_proba   = rf_pipe.predict_proba(X_test)[:, 1]
svm_proba  = svm_pipe.predict_proba(X_test)[:, 1]
lr_proba   = lr_pipe.predict_proba(X_test)[:, 1]
```

```python
# print results
print("\n--- Classification Results ---")
model_names = ['Random Forest', 'SVM', 'Logistic Regression']
all_preds   = [rf_pred, svm_pred, lr_pred]
all_probas  = [rf_proba, svm_proba, lr_proba]

results = []
for name, pred, proba in zip(model_names, all_preds, all_probas):
    results.append({
        'Model'    : name,
        'Accuracy' : round(accuracy_score(y_test, pred), 3),
        'Precision': round(precision_score(y_test, pred, zero_division=0), 3),
        'Recall'   : round(recall_score(y_test, pred, zero_division=0), 3),
        'F1 Score' : round(f1_score(y_test, pred, zero_division=0), 3),
        'ROC AUC'  : round(roc_auc_score(y_test, proba), 3),
    })

results_df = pd.DataFrame(results)
print(results_df.to_string(index=False))

# ==============================================================================
# STEP 6 - CONFUSION MATRICES FOR ALL MODELS
# ==============================================================================

print("\nGenerating Confusion Matrices...")

fig, axes = plt.subplots(1, 3, figsize=(18, 5))
fig.suptitle('Confusion Matrices for All Classification Models',
             fontsize=16, fontweight='bold')

for ax, name, pred in zip(axes, model_names, all_preds):
    cm = confusion_matrix(y_test, pred)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax,
                xticklabels=['No Coma (0)', 'Food Coma (1)'],
                yticklabels=['No Coma (0)', 'Food Coma (1)'],
                cbar=False, linewidths=0.5,
                annot_kws={'size': 14, 'weight': 'bold'})
    acc = accuracy_score(y_test, pred)
    f1  = f1_score(y_test, pred, zero_division=0)
    ax.set_title(f'{name}\nAccuracy={acc:.3f}  F1={f1:.3f}', fontsize=11, fontweight='bold')
    ax.set_xlabel('Predicted Label')
    ax.set_ylabel('True Label')

plt.tight_layout()
plt.show()

# ==============================================================================
# STEP 7 - ROC CURVES FOR ALL MODELS
# ==============================================================================

print("Generating ROC Curves...")

plt.figure(figsize=(8, 6))
```

```python

colors = ['#1565C0', '#E65100', '#2E7D32']
for name, proba, color in zip(model_names, all_probas, colors):
    fpr, tpr, _ = roc_curve(y_test, proba)
    auc_score   = roc_auc_score(y_test, proba)
    plt.plot(fpr, tpr, lw=2.5, color=color,
             label=f'{name}  (AUC = {auc_score:.3f})')

# diagonal line = random guessing baseline
plt.plot([0, 1], [0, 1], color='grey', lw=1.5, linestyle='--', label='Random Guess')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate', fontsize=12)
plt.ylabel('True Positive Rate', fontsize=12)
plt.title('ROC Curves — All Models', fontsize=14, fontweight='bold')
plt.legend(loc='lower right', fontsize=10)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# ==============================================================================
# STEP 8 - COMPREHENSIVE METRICS COMPARISON
# ==============================================================================

print("Generating Metrics Comparison Chart...")

plt.figure(figsize=(13, 6))
melted = results_df.melt(id_vars='Model', var_name='Metric', value_name='Score')
ax = sns.barplot(x='Metric', y='Score', hue='Model', data=melted,
                 palette=['#1565C0', '#E65100', '#2E7D32'])
plt.title('Comprehensive Evaluation Metrics — All Models', fontsize=14, fontweight='bold')
plt.ylabel('Score (0 to 1)')
plt.ylim(0, 1.15)
plt.legend(loc='lower right', fontsize=9)
for container in ax.containers:
    ax.bar_label(container, fmt='%.2f', padding=2, fontsize=8)
plt.tight_layout()
plt.show()

# ==============================================================================
# STEP 9 - FEATURE IMPORTANCE FOR ALL 3 MODELS
# ==============================================================================

# first we need the feature names after preprocessing (for RF and LR)
preprocessor.fit(X_train)
nom_cols = (preprocessor.named_transformers_['nom']
                         .named_steps['onehot']
                         .get_feature_names_out(nominal_features).tolist())
all_feat_names = numeric_features + ordinal_features + nom_cols

# clean up the names a bit for the charts
clean_names = {
    'How would you rate your sleep quality? '                    : 'Sleep Quality',
```

```python
    'How heavy did the meal feel?'                                : 'Meal Heaviness',
    'Meal Size'                                                   : 'Meal Size',
    'Carb Content'                                                : 'Carb Content',
    'Protein Content'                                             : 'Protein Content',
    'Stress level before eating'                                  : 'Stress Level',
    'What was the last meal type you had?_Breakfast'              : 'Meal: Breakfast',
    'What was the last meal type you had?_Dinner'                 : 'Meal: Dinner',
    'What was the last meal type you had?_Lunch'                  : 'Meal: Lunch',
    'What was the last meal type you had?_Snacks'                 : 'Meal: Snacks',
    'Physical activity before meal (last 2\u20133 hrs):_Intense'         : 'Activity: Intense',
    'Physical activity before meal (last 2\u20133 hrs):_Light (walking)' : 'Activity: Light',
    'Physical activity before meal (last 2\u20133 hrs):_Missing'         : 'Activity: Missing',
    'Physical activity before meal (last 2\u20133 hrs):_Moderate'        : 'Activity: Moderate',
}
display_names = [clean_names.get(f, f) for f in all_feat_names]

# ----------------------------------------------------------------
# 9A: Gini / MDI Importance  —  all 3 models in one figure
#     RF has a native .feature_importances_ (Gini / MDI).
#     LR and SVM don't, so we compute MDI-equivalent via
#     permutation importance on the *training* set (which mirrors
#     the MDI spirit of measuring impurity reduction on seen data).
# ----------------------------------------------------------------

short_feat_names = {
    'Meal Size'                                              : 'Meal Size',
    'Carb Content'                                           : 'Carb Content',
    'Protein Content'                                        : 'Protein Content',
    'How would you rate your sleep quality? '                : 'Sleep Quality',
    'What was the last meal type you had?'                   : 'Meal Type',
    'Physical activity before meal (last 2\u20133 hrs):'    : 'Pre-Meal Activity',
    'How heavy did the meal feel?'                           : 'Meal Heaviness',
    'Stress level before eating'                             : 'Stress Level',
}

# random forest has a built-in thing called feature_importances_ which basically
# tells us which features reduced the most impurity across all the trees
# its also called Gini importance or MDI (mean decrease in impurity)
# i learned that LR and SVM dont have this so we only plot it for RF here
rf_importances = rf_pipe.named_steps['classifier'].feature_importances_
rf_fi_df = pd.DataFrame({'Feature': display_names, 'Importance': rf_importances})
rf_fi_df = rf_fi_df.sort_values('Importance', ascending=True)

plt.figure(figsize=(9, 6))
plt.barh(rf_fi_df['Feature'], rf_fi_df['Importance'],
         color='#1565C0', edgecolor='black', alpha=0.85)
plt.title('Random Forest — Gini / MDI Feature Importance', fontsize=14, fontweight='bold')
plt.xlabel('Mean Decrease in Gini Impurity')
for i, v in enumerate(rf_fi_df['Importance']):
```

```python
    plt.text(v + 0.001, i, f'{v:.4f}', va='center', fontsize=7)
plt.tight_layout()
plt.show()

# ----------------------------------------------------------------
# 9B: Permutation Feature Importance on TEST set — all 3 models
# ----------------------------------------------------------------

fig2, axes2 = plt.subplots(1, 3, figsize=(22, 6))
fig2.suptitle('Permutation Feature Importance (Test Set) — All 3 Models',
              fontsize=16, fontweight='bold')

perm_colors = ['#1565C0', '#E65100', '#2E7D32']
pipes       = [rf_pipe,   lr_pipe,   svm_pipe]
pipe_labels = ['Random Forest', 'Logistic Regression', 'SVM']

for ax, pipe, label, color in zip(axes2, pipes, pipe_labels, perm_colors):
    perm_res = permutation_importance(pipe, X_test, y_test,
                                      n_repeats=15, random_state=42, scoring='f1')
    perm_df_test = pd.DataFrame({
        'Feature'   : selected_features,
        'Importance': perm_res.importances_mean,
        'Std'       : perm_res.importances_std
    })
    perm_df_test['Feature'] = perm_df_test['Feature'].map(short_feat_names)
    perm_df_test = perm_df_test.sort_values('Importance', ascending=True)

    ax.barh(perm_df_test['Feature'], perm_df_test['Importance'],
            xerr=perm_df_test['Std'], color=color, edgecolor='black',
            alpha=0.85, capsize=4)
    ax.axvline(0, color='black', lw=0.8, linestyle='--')
    ax.set_title(f'{label}\n(Permutation Importance on Test Set)',
                 fontweight='bold', fontsize=11)
    ax.set_xlabel('Mean Decrease in F1 Score')

plt.tight_layout()
plt.show()

# ==============================================================================
# STEP 10 - FEATURE CORRELATION HEATMAP
# ==============================================================================

print("Generating Feature Correlation Heatmap...")

# transform the training data so we can compute correlations
X_train_transformed = preprocessor.fit_transform(X_train)
X_train_df = pd.DataFrame(X_train_transformed, columns=display_names)

plt.figure(figsize=(14, 11))
corr_matrix = X_train_df.corr()
# mask the upper triangle so it doesn't repeat
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))
sns.heatmap(corr_matrix,
            mask=mask,
```

```python
            cmap=sns.diverging_palette(230, 20, as_cmap=True),
            vmax=1.0, vmin=-1.0, center=0,
            annot=True, fmt='.2f', annot_kws={'size': 8},
            square=True, linewidths=0.5,
            cbar_kws={'shrink': 0.6, 'label': 'Pearson r'})
plt.title('Feature Correlation Heatmap (Transformed Training Data)',
          fontsize=14, fontweight='bold', pad=15)
plt.tight_layout()
plt.show()

# ==============================================================================
# STEP 11 - SHAP ANALYSIS (only works if shap is installed)
# ==============================================================================

if shap_available:
    print("Generating SHAP plots...")

    # TreeExplainer works well with Random Forest
    explainer   = shap.TreeExplainer(rf_pipe.named_steps['classifier'])
    shap_values = explainer.shap_values(X_train_df)

    # shap_values is a list of arrays for each class - we want class 1 (Drowsy)
    if isinstance(shap_values, list):
        sv = shap_values[1]
    elif len(shap_values.shape) == 3:
        sv = shap_values[:, :, 1]
    else:
        sv = shap_values

    # --- SHAP Plot 1: Mean SHAP Value Bar Plot ---
    plt.figure(figsize=(9, 6))
    shap.summary_plot(sv, X_train_df, plot_type='bar', show=False)
    plt.title('Mean SHAP Value Plot — Global Feature Importance',
              fontsize=14, fontweight='bold')
    plt.tight_layout()
    plt.show()

    # --- SHAP Plot 2: Beeswarm Plot ---
    plt.figure(figsize=(9, 6))
    shap.summary_plot(sv, X_train_df, show=False)
    plt.title('SHAP Beeswarm Plot — Direction of Feature Impact\n'
              '(Red = high feature value, Blue = low feature value)',
              fontsize=13, fontweight='bold')
    plt.tight_layout()
    plt.show()

else:
    print("Skipping SHAP — install it with: pip install shap")

print("\nDone! All plots generated.")
```